Hackage: https://hackage.haskell.org/package/optparse-applicative
Introduction: https://github.com/pcapriotti/optparse-applicative

```
-- Options.Applicative
data Parser a
instance Functor Parser, Applicative Parser, Alternative Parser


data OptionFields a                                             --
option with value e.g. --config PATH
data FlagFields a                                               --
option with no value e.g. --recursive
data ArgumentFields a                                           --
positional params
data CommandFields a

class HasName f                                                 --
FlagFields, OptionFields
class HasCompleter f                                            --
ArgumentFields, OptionFields
class HasValue f                                                --
ArgumentFields, OptionFields
class HasMetavar f                                              --
ArgumentFields, CommandFields, OptionFields


-- Arguments aka position parameters
argument :: ReadM a -> Mod ArgumentFields a -> Parser a
strArgument :: IsString s => Mod ArgumentFields s -> Parser s
flag :: a -> a -> Mod FlagFields a -> Parser a
flag' :: a -> Mod FlagFields a -> Parser a
switch :: Mod FlagFields Bool -> Parser Bool
option :: ReadM a -> Mod OptionFields a -> Parser a
strOption :: IsString s => Mod OptionFields s -> Parser s

data ReadM a
auto :: Read a => ReadM a
str :: IsString a => ReadM a
maybeReader :: (String -> Maybe a) -> ReadM a
eitherReader :: (String -> Either String a) -> ReadM a
disabled :: ReadM a


-- Modifier
data Mod f a
instance Semigroup (Mod f a), Monoid (Mod f a)

short :: HasName f => Char -> Mod f a
long :: HasName f => String -> Mod f a
metavar :: HasMetavar f => String -> Mod f a
value :: HasValue f => a -> Mod f a
showDefault :: Show a => Mod f a
```

```
showDefaultWith :: (a -> String) -> Mod f a
hidden :: Mod f a
internal :: Mod f a
help :: String -> Mod f a
helpDoc :: Maybe Doc -> Mod f a
style :: (Doc -> Doc) -> Mod f a


--  Commands
command :: String -> ParserInfo a -> Mod CommandFields a
commandGroup :: String -> Mod CommandFields a
subparser :: Mod CommandFields a -> Parser a
hsubparser :: Mod CommandFields a -> Parser a


-- Parser extensions
abortOption :: ParseError -> Mod OptionFields (a -> a) -> Parser (a
-> a)
infoOption :: String -> Mod OptionFields (a -> a) -> Parser (a ->
a)
helper :: Parser (a -> a)


-- Program info
info :: Parser a -> InfoMod a -> ParserInfo a
fullDesc :: InfoMod a
briefDesc :: InfoMod a
progDesc :: String -> InfoMod a
failureCode :: Int -> InfoMode a


-- Execution
data ParserResult a =
 Success a |
 Failure (ParserFailure ParserHelp) |
 CompletionInvoked CompletionResult

execParser :: ParserInfo a -> IO a
customExecParser :: ParserPrefs -> ParserInfo a -> IO a
execParserPure :: praserPrefs -> ParserInfo a -> [String] ->
ParserResult a
defaultPrefs :: ParserPrefs
```

optparse-applicative
A module for parsing commandline options

e.g. Basic usage

```
myparser :: Parser (String, Bool, Int)
myparser = (,,)
  <$> strArgument (metavar "ARGS")             -- arguments
  <*> switch (short 't')                       -- -t boolean flag
```

```
  <*> option auto (short 'i' <> long "int")     -- (-i|--int)
option

main :: IO ()
main = do
  args <- execParser $ info (helper <*> myparser) (fullDesc <>
progDesc "My program")
  print args
```

Command help

```
$ main --help
Usage: main ARGS [-t] (-i|--int ARG)

  My program

Available options:
  -h,--help                Show this help text
  ARGS                     input values
  -t                       some flag
  -i,--int ARG             some int
```

Command with various inputs

```
$ main hello               # arguments
(hello, False, 0)

$ main -t hello            # -t flag
(hello, True, 0)

$ main -i 10 hello         # -i option
$ main --int 10 hello
(hello, False, 10)

$ main --int 10 -t hello    # all
(hello, True, 10)

$ main                     # ERROR. ARGS is required
$ main -a                  # ERROR. Unknown option/flag
```

```
---- Basics
--
switch (short 'a') :: Parser Bool                        -- parse a
single value
(,,)                                                     -- parse
an n-ary value
  <$> strArugment (metavar "VALUES")
  <*> switch (short 'a')
  <*> switch (short 'b') :: Parser (String, Bool, Bool)
```

```
-- Parser: argument
--
argument :: ReadM a -> Mod ArgumentFields a -> Parser a
strArgument :: IsString s => Mod ArgumentFields s -> Parser s
strArgument (metavar "VALUES") :: Parser String
     -- required arguments
strArgument (metavar "VALUES" <> value "") :: Parser String
     -- optional with default ""
optional (strArgument (metavar "VALUES")) :: Parser (Maybe String)
    -- optional with default Nothing


-- Parser: flag
-- i.e. option with on/off
--
flag :: a -> a -> Mod FlagFields a -> Parser a
flag' :: a -> Mod FlagFields a -> Parser a
switch :: Mod FlagFields Bool -> Parser Bool
flag 0 1 (short 'f') :: Parser Int                          --
optional flag -f with default 0 active 1
flag' 1 (short 'f') :: Parser Int                           --
required
optional (flag' 1 (short 'f')) :: Parser (Maybe Int)      --
optional with default Nothing
switch (short 'f') :: Parser Bool                           --
optional with default False
flag False True (short 'f') :: Parser Bool                  -- same


-- parser: options
-- i.e. option with values
--
option :: ReadM a -> Mod OptionFields a -> Parser a
strOption :: IsString s -> Mod OptionFields s -> Parser s
option auto (short 'b') :: Parser Int                        --
required option -b <number>
option auto (short 'b' <> value 0) :: Parser Int            --
optional with default 0
optional (option auto (short 'b')) :: Parser (Maybe Int)   --
optional with default Nothing


-- parser: command
--
subparser :: Mod CommandFields a -> Parser a
hsubparser :: Mod CommandFields a -> Parser a
command :: String -> ParserInfo a -> Mod CommandFields a
commandGroup :: String -> Mod CommandFields a
p, q :: Parser a
subparser (
 command "start" (info p (ProgDesc "Start command"))
 <> command "stop" (info p (ProgDesc "Stop command")))
```

```
$ main -h
Usage: main [COMMAND]

     Available options:
       -h,--help                Show this help text

     Available commands:
       start                    Start command
       stop                     Stop command


$ main start -h
Usage: main start [-f]

     Available options:
       -f                       My flag
       -h,--help                Show this help text
```

```
---- parser: extensions
-- i.e. extend parser with additional behavior
--
-- helper: add -h|--help option
helper :: Parser (a -> a)
helper <*> parser
```

```
$ main -h
$ main --help

Usage: main [-h] ...
```

```
-- aborting: i.e. options that always fail
abortOption :: ParseError -> Mod OptionFields (a -> a) -> Parser (a
-> a)
infoOption :: String -> Mod OptionFields (a -> a) -> Parser (a ->
a)
abortOption (ErrorMsg "NOT IMPLEMENTED) (short 'i') <*> parser
```

```
$ main -i
NOT IMPLEMENTED

Usage: main [-i] ...

$ echo $?                     -- fail
1
```

```
infoOption "Aborting" (short 'i') <*> parser
```

```
$ main -i
Aborting
```

```
$ echo $?                       -- success
0
```

Modifiers

Modifier

Settings for each command option

e.g.

```
--  [-n|---name NAME] option with default value "NA"
strOption (short 'n' <> long "name" <> value "NA" <>
metavar "NAME")
```

```
---- Basics
short 'c'                          -- single
short 'c' <> long "cmd"            -- combine

-- options
short :: HasName f => Char -> Mod f a
long :: HasName f => String -> Mod f a
metavar :: HasMetavar f => String -> Mod f a
value :: HasValue f => a -> Mod f a
```